```
def change_values
@a += 1
@@a += 1
end
end

eg = BlockExample.new
block = eg.return_closure { "original" }

block.call # => [0, 1, 2, 3, "original"]
eg.change_values
block.call # => [0, 1, 3, 4, "original"]
```

Here, the return_closure method returns a lambda that encapsulates access to the local variable a, instance variable @a, class variable @@a, and constant CONST. We call the block outside the scope of the object that contains these values, but they're still available via the closure. If we call the object to change some values, the values accessed via the closure also change.

Block Arguments

Block argument lists are very similar to method argument lists:

- You can specify default values.
- You can specify splat (starred) arguments.
- You can specify keyword arguments
- You can specify a double-splat argument
- The last argument can be prefixed with an ampersand, in which case it will collect any block passed when the original block is called.
- Block-local variables are declared after a semicolon in the argument list.
- You can't use the anonymous forwarding syntaxes, *, **, &, or ....

Within a block you can avoid giving positional arguments names and instead refer to them by their numerical position using _1, _2, and so on.

```
[1, 2, 3].map { _1.sqrt }
```

Using both named positional arguments and numerical ones is a syntax error. Naming a local variable using the _1 pattern is also a syntax error.

Proc Objects

Ruby's blocks are chunks of code attached to a method. Blocks are not objects, but they can be converted into objects of class Proc. There are many ways to convert a block into a Proc object.

- By passing a block to a method whose last parameter is prefixed with an ampersand. That parameter will receive the block as a Proc object.

```
def meth1(p1, p2, &block)
  puts block.inspect
end

meth1(1,2) { "a block" }
meth1(3,4)
```